

SEEING THE PATTERNS, CLOSING THE LOOP: EVOLVING ENVIRONMENTS FOR AGENT TRAINING

Anonymous authors

Paper under double-blind review

ABSTRACT

LLM agents can serve as data researchers, automating the creation of training data for LLM post-training. We study data-centric recursive self-improvement for agent post-training. In this setting, a researcher agent starts from an empty environment library and constructs environments for online RL without access to the target backend or verifier; evaluation of the updated model then supplies feedback for the next round. Our experiments and audits identify three challenges: incomplete behavioral analysis, disconnected data–training–evaluation feedback, and incomplete implementation of intended environment mechanisms. We introduce `EnvOpt`, which uses source-traceable hierarchical analysis to *see the patterns* in agent behavior and target-conditioned context refinement to *close the loop* between data, training, and evaluation. These two designs guide environment construction and revision, supported by specification-driven implementation and independent review. Preliminary experiments with an earlier implementation yield observed test gains of 16.67 and 11.11 percentage points on BFCL and ACEBench, respectively, although gains vary across rounds. These findings suggest that evidence-guided environment evolution is a promising direction for automated agent post-training.

1 INTRODUCTION

Data-centric recursive self-improvement (RSI) places an agent in the role of a data researcher: inspect a model’s behavior, construct training experience, train the model, and use the resulting feedback to decide what to try next. RSIBench-Data studies this cross-model research setting with a fixed researcher and controlled training and evaluation services (Meng et al., 2026). Its discovery–reliability gap motivates looking beyond whether an agent can find a useful dataset once. The harder question is how it should conduct the next round of research as both the model and its training data change.

We study this question for tool-using models, with permitted target tasks and interaction trajectories but without access to the target backend or verifier implementation. The training–environment library starts empty. The researcher must construct independent executable environments that supply state, tool effects, new task instances, and outcome checks. The target model trains in these environments with online RL, and its updated evaluation trajectories inform the next environment-generation round. This is a from-scratch environment-research constraint, not an absence of model priors, generic interface examples, or evidence from the public eval set. Executable synthesis and environment adaptation already provide substantial foundations (Song et al., 2026; Wang et al., 2026; Huang et al., 2026; Liu et al., 2026; Fang et al., 2026); our focus is how to organize the evidence that guides their repeated use.

Across this workflow, we identify three challenges: *incomplete behavioral analysis* can omit task requirements and failure mechanisms; *disconnected data, training, and evaluation* obscure which intervention was actually tested; and *incomplete environment implementation* can omit mechanisms required by the training target. Section 3 examines these through report-coverage measurements, a training-exposure case, and a generated-program audit. Figure 1 previews the two evidence gaps addressed by our core designs.

We present `EnvOpt`, organized around two complementary designs. A *traceable behavioral map* combines per-trajectory analysis, hierarchical merging with source links, and measured task-linked

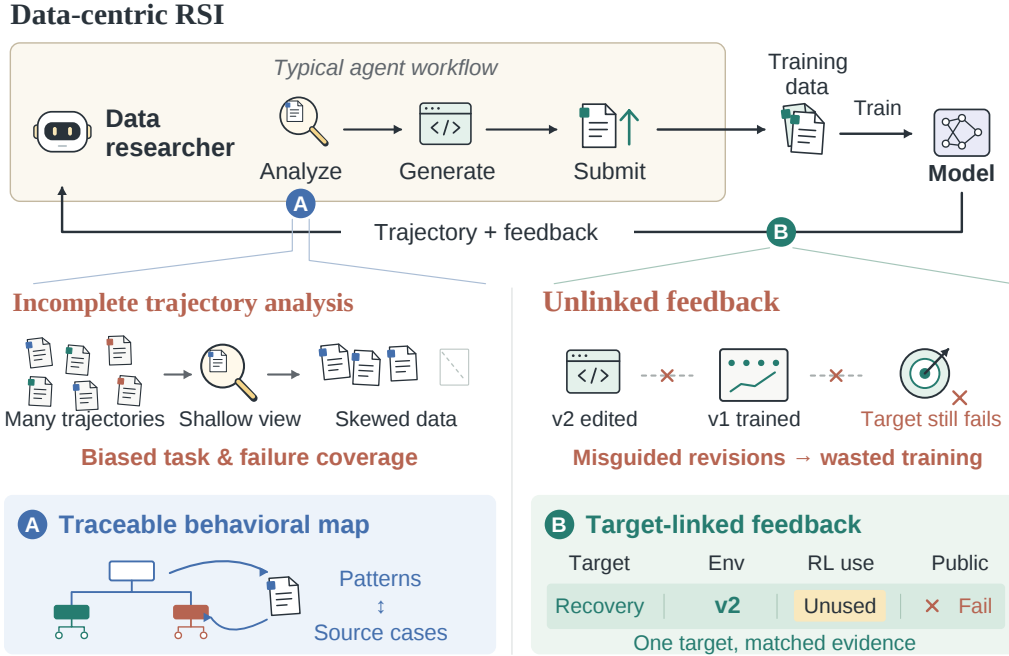


Figure 1: Data-centric RSI faces incomplete trajectory analysis (A) and disconnected data–training–evaluation feedback (B). Illustrations and records are schematic.

requirement groups. The data researcher can survey behavioral patterns and return to the original cases, rather than treating a compressed interpretation as ground truth. *Target-conditioned context refinement* targets an *information-utilization gap*: access to feedback does not ensure its use in research decisions. Target-linked feedback organizes intended training, actual RL experience, and subsequent target outcomes around a shared behavioral target to form *decision-ready context*.

A persistent data researcher owns the research interpretation and expected effects. Parallel, persistent Env Builders implement its specifications and examples, refining their programs through independent review. This execution workflow realizes the two designs; program generation, environment memory, and validation are not claimed as separate inventions. We also do not treat the bottlenecks as a measured universal failure of native coding agents: the relevant comparison must match information access and research and training budgets.

The training evidence comprises preliminary three-benchmark Qwen3-4B studies of an earlier implementation. BFCL improves substantially, while τ^2 regresses after an intermediate peak and ACEBench fluctuates on a small private eval set. These observations motivate tracking the full research trajectory rather than reporting only the best checkpoint. They do not yet isolate the two designs or validate the latest persistent-builder revision; those claims require the controlled comparisons specified in Section 5. The separate analysis diagnostic evaluates report quality on frozen Qwen3-8B trajectories, not the cause of these training gains.

2 RELATED WORK

Data-centric RSI. RSIBench-Data studies agents that revise training-data strategies for a separate target model under fixed training, evaluation, and resource controls (Meng et al., 2026). Autodata develops an agentic data scientist that iterates between data creation and quality assessment, including meta-optimization of the data scientist itself (Kulikov et al., 2026). DataMaster combines data discovery and transformation with tree search and cumulative memory while holding the learning algorithm fixed (Du et al., 2026). EnvOpt studies this research role for executable training environments: a researcher starts with an empty environment library and uses behavioral evidence to guide

successive rounds of environment revision and online RL. Its focus is the evidence supporting those research decisions.

Executable environment synthesis for tool-calling agents. AgentScaler, EnvScaler, and ScaleEnv automate the construction of executable tools, state, and verifiable tasks (Fang et al., 2025; Song et al., 2026; Tu et al., 2026). Agent World Model scales database-backed synthetic environments for RL, while EnvFactory combines executable environment construction with topology-aware task synthesis (Wang et al., 2026; Xu et al., 2026). FACET grounds task instructions, solutions, and verifiers in shared executable state to preserve source intent and cross-artifact consistency (Shi et al., 2026). The ACE framework and WRIT further examine data accuracy, learner-relative complexity, diversity, and the evidence required for individual tool-use decisions (Zeng et al., 2026; Gu et al., 2026). These works establish construction and validation principles that EnvOpt builds on; our emphasis is using source-traceable behavioral analysis and actual-training feedback to decide which mechanisms to construct or revise.

Iterative RL and environment adaptation. PAIRED and Prioritized Level Replay adapt environment generation or selection using regret and learning-potential signals (Dennis et al., 2020; Jiang et al., 2021). For tool-use agents, RODS uses online reward statistics to synthesize structurally aligned tasks and update a replay pool (Fang et al., 2026). EnvHarness adapts existing environments from policy trajectories while retaining their original verifiers (Huang et al., 2026). SPADE trains a shared environment-designer and solver model through self-play, using hint-based regret, corpus grounding, and environment memory (Liu et al., 2026). Agent-World and AgentOmnia also use behavioral diagnosis to guide targeted task synthesis and self-evolution (Dong et al., 2026; Jiang et al., 2026). EnvOpt uses an external researcher to revise independent environments without the target backend or verifier. It connects the source evidence behind a diagnosis to realized program versions, training tasks, RL observations, and fixed-target evaluation, supporting inspection of what each intervention actually delivered.

3 AGENTS AS DATA RESEARCHERS: IDENTIFYING CHALLENGES IN DATA-CENTRIC RSI

We instantiate data-centric RSI as an iterative environment-generation and reinforcement-learning loop. An agent serves as the data researcher, constructing executable environments and tasks in which a target model trains with online RL. Feedback from training and evaluation informs the next round. We first formalize this process, then examine challenges in its research context and generated training instances.

Formalizing the data-research loop. A *research context* is the information available to an agent for research decisions, encompassing the research history, current state, and other decision-relevant information. Let C_t denote this context at round t and DataResearcher the agent that uses it to construct training instances. Each instance is a triple (e, x, v) , consisting of an executable environment e , a task x specifying a goal and initial conditions, and a verifier v assessing task completion. The agent’s output is a set D_t of n_t such instances:

$$D_t = \{(e_j, x_j, v_j)\}_{j=1}^{n_t} = \text{DataResearcher}(C_t). \quad (1)$$

The set D_t is then submitted for training and evaluation. Let π_t denote the target model at the start of round t and TrainEval the fixed online-RL and public-evaluation procedure. It returns the updated model π_{t+1} and feedback F_t comprising training records and evaluation results:

$$(\pi_{t+1}, F_t) = \text{TrainEval}(\pi_t, D_t). \quad (2)$$

Finally, UpdateContext integrates the submitted set and its feedback with the prior research context to guide the next round:

$$C_{t+1} = \text{UpdateContext}(C_t, D_t, F_t). \quad (3)$$

In this setting, the loop starts from an empty environment library. Guided by feedback and the evolving research context, the data researcher repeatedly creates new environments or revises existing ones, with the goal of improving the target model over successive training rounds.

3.1 INCOMPLETE BEHAVIORAL ANALYSIS

The research context must preserve task requirements and model behavior to support training-target selection. To examine what survives this analysis, we give an analyst agent (Astra/high) complete access to the initial public-eval trajectories of Qwen3-8B reasoning: 1,200 BFCL, 417 τ^2 , and 75 ACEBench-Agent trajectories. The instruction requests an analysis and specifies report fields, while leaving the reading strategy to the agent. We measure report coverage of source-grounded task requirements and failure mechanisms, crediting full preservation of their consequential conditions and dependencies (Section 5.3).

Figure 2 shows the resulting gap: task-requirement coverage is 61.5–82.5%, while failure-mechanism coverage ranges from only 39.4% on BFCL to 67.5% on ACE. These automatic, task-averaged scores use the same final paired evaluation as Section 5.3; the remainder includes both partial descriptions and wholly missing mechanisms. Full-corpus access thus leaves substantial behavioral information incompletely represented. Coverage is lower on the larger corpora in this comparison, although benchmark composition and corpus size vary together, preventing a causal size claim.

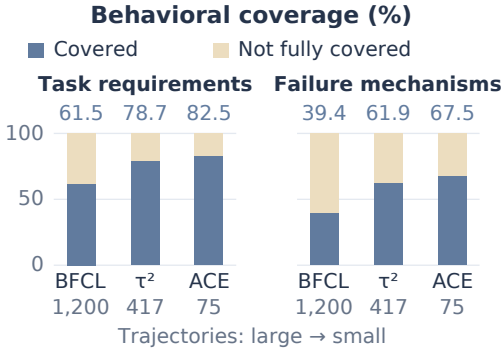


Figure 2: Report coverage, ordered by trajectory count. Same scores as Section 5.3; upper segments include partial coverage.

The research context therefore needs to preserve the mechanisms and conditions that matter for the next data decision, with evidence available for inspection.

3.2 DISCONNECTED DATA, TRAINING, AND EVALUATION

Updating the research context requires connecting intended training targets, tasks actually trained, and subsequent evaluation outcomes. In ACE R9, one revision contributed zero of its ten planned tasks. We further audited 28 iterations of the original 4B studies, measuring the fraction of trained tasks with nonconstant rewards across eight valid rollouts—the within-task contrast used by GRPO. The task-weighted rates were 74.5%, 41.9%, and 37.2% on τ^2 , BFCL, and ACE. These checks distinguish whether an intervention reached training and whether its rollouts supplied reward contrast, motivating explicit linkage to its intended target and evaluation outcomes. [TODO: Insert Codex per-iteration results and the learning-signal comparison.]

3.3 INCOMPLETE ENVIRONMENT IMPLEMENTATION

A retail-exchange environment required matching identical items to different replacements, yet all six inspected tasks assigned the same replacement to both items. Checking the generator against the specification confirmed that the required branch was absent (Appendix B.1). This exposes a gap in constructing the output triples: a runnable instance need not realize the intended training mechanism. Auditing environment behavior, task requirements, and verifier scores tests whether the three components jointly deliver the training target. [TODO: Cross-benchmark audit: 20 environments each on τ^2 , BFCL, and ACE; eight rollouts per environment; LLM judgments of target implementation, interaction, and verifier defects. Insert observed defect rates.]

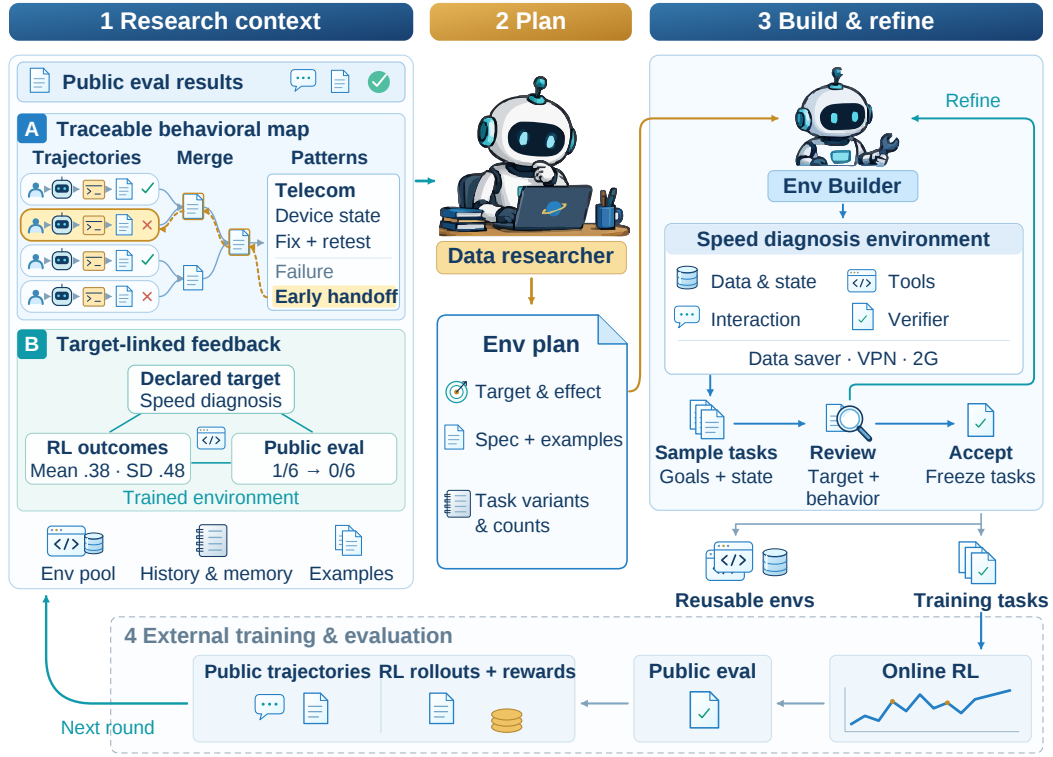


Figure 3: EnvOpt’s research loop. A highlighted pattern traces back through the merge tree; linked target, training, and public-eval evidence guide the data researcher. Sampled tasks are independently reviewed and frozen before RL. Curves are schematic; B shows one historical example.

4 METHOD

EnvOpt instantiates the data-research loop in Equations 1–3 (Figure 3). Its two core designs organize the research context C_t : a *traceable behavioral map* preserves task requirements and failure mechanisms with links to source trajectories; *target-conditioned context refinement* uses target-linked feedback to build decision-ready context for comparing intended training, actual RL experience, and target-evaluation outcomes. Guided by this context, specification-driven construction and independent review produce the environment–task–verifier set D_t . The training-and-evaluation service remains fixed.

The following describes the maintained framework and labels a standalone analysis refinement. Section 5 and Appendix B distinguish these from the earlier protocols underlying the reported training curves.

4.1 TRACEABLE BEHAVIORAL MAP

The map separates a persistent evidence layer from its compact reading view. Let $T_{t,i}$ contain the complete public-eval episodes for task i , including instructions, tool definitions, and observed interactions. Extraction and source review produce an immutable mechanism ledger:

$$\mathcal{L}_t = \bigcup_i \text{ExtractReview}(T_{t,i}). \quad (4)$$

Each record links an environment mechanism, task requirement, or observed failure/recovery to its source and applicable conditions.

Bounded hierarchical merging builds a draft by factoring shared premises while retaining distinct actors, scopes, dependencies, and recovery branches:

$$R_t^{(0)} = \text{FactorMerge}(\mathcal{L}_t). \quad (5)$$

The draft is checked against the retained evidence, not only earlier summaries. Repair locally corrects conflicting claims; Residual extracts and consolidates only relations still missing from the corrected report:

$$\begin{aligned} \bar{R}_t &= \text{Repair}(R_t^{(0)}, \mathcal{L}_t), \\ R_t &= \bar{R}_t \oplus \text{Residual}(\mathcal{L}_t, \bar{R}_t). \end{aligned} \quad (6)$$

Here $\oplus$ appends the missing distinctions without rewriting preserved content; an already-covered record must instead be matched to an existing passage. These are model-based checks, not a semantic-completeness guarantee.

The report retains the four fields *domain*, *environment*, *task behavior*, *failure behavior*. It enters the research context C_t with links back to original trajectories. Task-linked measurement and construction details are given in Appendix E. This coverage-checked variant is evaluated offline in Section 5.3, separately from the earlier analysis used in the historical training runs.

4.2 TARGET-CONDITIONED CONTEXT REFINEMENT

Access to evidence does not ensure its use in research decisions. We target this *information-utilization gap* with *decision-ready context*: evidence organized for joint interpretation around a shared behavioral target. Our *target-linked feedback* provides the context-level intervention, explicitly connecting what was intended, what was trained, and how target behavior changed.

For each intervention, the researcher declares a target requirement and expected effect, anchored to public-eval tasks. Versioned triples in D_t and their task IDs link this target to actual RL groups and rollouts, forming a target-conditioned evidence view:

$$\underbrace{\text{Training target}}_{\text{requirement, expected effect}} \longleftrightarrow \underbrace{\text{Actual training}}_{\text{versioned triples, RL rollouts}} \longleftrightarrow \underbrace{\text{Target evaluation}}_{\text{linked public tasks}}. \quad (7)$$

The view distinguishes planned, accepted, and actually trained data, pairing training-time reward statistics and within-task reward contrast with linked public-eval outcomes and trajectories. The researcher compares these observations with the expected effect, recording agreements, mismatches, and unresolved questions in C_{t+1} . This assessment guides revisions to mechanisms, challenge, or sampling, and decisions to retain or pause environments. The design guides evidence use rather than adding otherwise inaccessible feedback; the links do not establish causal effects of individual edits.

4.3 ITERATIVE ENVIRONMENT CONSTRUCTION

We construct training environments through an iterative cycle of planning, implementation, and review, revising either the implementation or the plan as needed. The researcher selects training targets and task allocations, then provides specifications with success, failure, and recovery cases. Persistent Env Builders receive these specifications rather than raw public-eval or RL corpora.

Each builder implements candidate (e, x, v) triples, with task requests and verifiers sharing the same goal data. Execution checks and independent review examine the specification, full program, and all tasks in the batch. They assess whether the intended decisions and variations are implemented and whether information access, tool effects, and verifier rewards are consistent. Implementation findings return to the same builder for revision and renewed review, retaining prior findings; specification ambiguities return to the researcher to revise the plan.

Accepted triples are frozen and retained while the researcher replans any remaining allocations, subject to construction budgets. The accepted set D_t enters training once its admission requirement is met. This local construction cycle precedes the outer training and feedback loop; review remains fallible, not a proof of program correctness. Construction and acceptance details appear in Appendix A.

5 EXPERIMENTS

5.1 EXPERIMENTAL SETUP

Evaluation and training. We evaluate on τ^2 -Bench (Barres et al., 2026), BFCL-v4 (multi-turn) (Patil et al., 2025), ACEBench-Agent (Chen et al., 2025), and AppWorld (Trivedi et al., 2024). The first three benchmarks use 1:1 public/private splits; AppWorld uses its train split for public evaluation and test-normal split for private evaluation. Public evaluation informs environment evolution and checkpoint selection, while private evaluation is withheld from the evolution process and used for final performance comparisons. Target models train with GRPO (Shao et al., 2024). Benchmark protocols, training configurations, and implementation versions are given in Appendix B.

Implementation and baselines. Our target models are Qwen3-4B-Instruct-2507 and Qwen3-8B (Yang et al., 2025), with reasoning enabled for the latter. Our data researcher agent, DataResearcher, is instantiated with Codex powered by GPT-6 Astra. We grant the agent access to the GPT-5.5 API and 4 A100 GPUs during evolution. We compare against three baselines: vanilla Codex as an autonomous data researcher (Codex-Vanilla); Codex with an example code repository provided (Codex-E); and training on a fixed environment set generated by DataResearcher without revision (Fixed-Env). All researcher agents in these baselines are powered by GPT-6 Astra.

5.2 MAIN RESULTS

Limited sustained gains with a strong research agent. Despite pairing GPT-6 Astra with the Codex harness, Codex-Vanilla shows early gains followed by saturation or regression (Figure 4). On BFCL, public performance peaks at 30.00% in R3 and falls to 25.42% by R10. On τ^2 , it finishes at 21.58%, below its 27.10% starting score.

A plausible bottleneck is maintaining the learning value of generated environments as the target model changes. This requires updating training targets from behavioral feedback and faithfully implementing the corresponding mechanisms, the challenges identified in Section 3. Incomplete implementations or poorly updated targets may instead produce executable environments that offer little additional learning signal.

Substantial gains across four benchmarks. EnvOpt improves 4B public-evaluation performance by +8.63, +22.58, +30.00, and +26.75 percentage points over the base model on τ^2 -Bench, BFCL-v4 (multiturn), ACEBench-Agent, and AppWorld, respectively. These results use the earliest public maximum within the displayed rounds. The contrast is clearest on BFCL: EnvOpt reaches 45.00% in R9, well beyond Codex-Vanilla’s 30.00% peak in R3. On both benchmarks with baseline curves, EnvOpt maintains higher scores throughout the shared R3–R9 rounds. Progress is not monotonic, particularly on τ^2 after R5, but all four displayed endpoints remain above their starting scores. Private-evaluation results are reported in Appendix B.

5.3 BEHAVIORAL COVERAGE AND REPORT COMPACTNESS

We now test whether a structured analysis process reduces the information loss identified in Section 3.1. On the same 1,692 frozen initial Qwen3-8B public-eval trajectories, we compare the fixed coding-agent reports (Astra/high) with a standalone, coverage-checked refinement of the behavioral map (GPT-5.5/high). The refinement analyzes complete task evidence, consolidates shared mechanisms, and checks the compact report for omitted conditions. It receives neither the reference labels nor the coding agent’s report prose.

Measure retained meaning, not compression alone. Frozen, source-grounded reference units describe task requirements and failure mechanisms. An LLM judge scores anonymized report pairs as full, partial, missing, or contradicted. Only full preservation of the mechanism and its consequential scope or dependency receives credit. We average units within each task and then tasks within each corpus; word count separately measures compactness. References are generated before seeing candidate reports, not inferred from either method’s preferred categories. Appendix E gives the protocol, grading variability, costs, and spot audit.

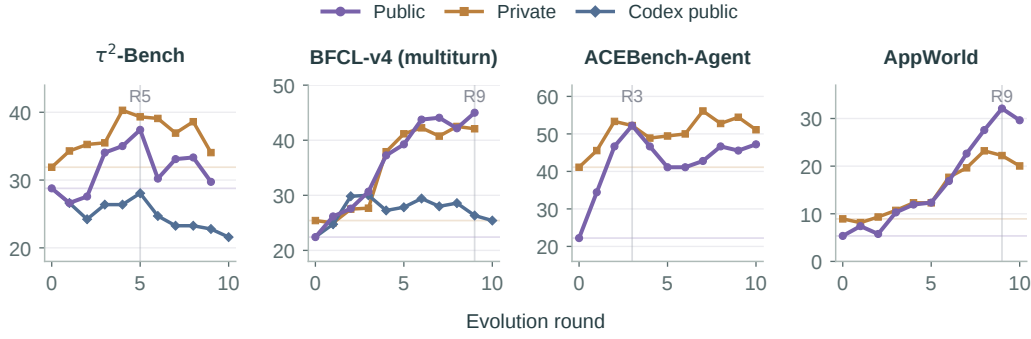


Figure 4: Public- and private-evaluation performance across environment-evolution rounds on four benchmarks, with Codex public-evaluation curves on τ^2 and BFCL.

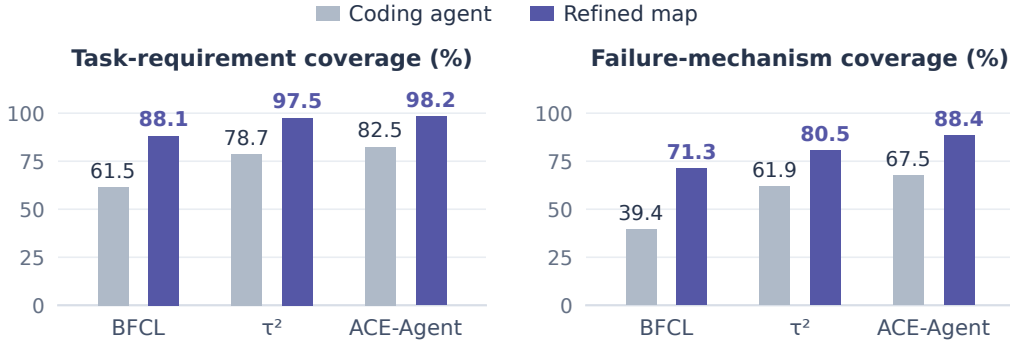


Figure 5: Task and failure coverage of the coding-agent reports and refined map under the final paired evaluation. Report lengths and historical grades are retained in Appendix E.

Higher coverage in shorter reports. The refined map reaches 88.1–98.2% task-requirement coverage and 71.3–88.4% failure-mechanism coverage, while using 12–29% fewer words (Figure 5). It exceeds both the paired and original grades of the unchanged coding-agent reports on all three corpora; the ordering also holds under subset–family-balanced aggregation. A concrete ACE example shows why this is more than naming more error types: an inbox-cleanup timeout label omits the user-authorization–deletion–retry dependency, which the refined report preserves. Both graders support partial versus full coverage for that example. Not every case improves; partially preserved mechanisms and other collapsed distinctions are documented in Appendix E.

These development-set results support more comprehensive, compact behavioral analysis in this setting. They do not isolate the hierarchy from the added coverage checks, match backbones or computation, or establish downstream RL gains. The diagnostic is separate from the historical 4B training runs.

5.4 REMAINING CONTROLS AND UNCERTAINTY

Controls still required. The available baseline records cover Codex–Vanilla; Codex–E and Fixed–Env comparisons are not yet reported. The offline diagnostic measures report quality, not a budget-matched isolation of the two designs. Such controls should match researcher, information, and training budgets. For behavioral understanding, compare selective case reading, a flat summary, and the source-traceable hierarchy, counting both analysis and researcher tokens. For context refinement, compare the same underlying records with and without target-linked organization, measuring evidence use and subsequent decisions under matched budgets and construction workflows. The proposed attention-guidance effect has not been isolated. An end-to-end native coding-agent control should retain equivalent tools and service access; the separately reported Codex curves do

not establish such a matched control. Secondary controls compare a fixed library with fresh tasks, sampler/allocation-only adaptation, and program revision, or remove maintained experience. These causal controls remain unmeasured. SPADE-style or EnvHarness-style controls must additionally disclose differences in target-code and public-eval-feedback access.

Uncertainty and cost. Paired-task uncertainty is estimated using 10,000 bootstrap resamples of private-eval tasks within fixed subsets, keeping each task’s three repeats together and preserving the benchmark aggregation. These intervals measure finite-private-eval-set uncertainty conditional on a trained run, not training-seed variability. The recorded rounds contain 846, 809, and 626 distinct training Tasks for τ^2 , BFCL, and ACE respectively, with eight episodes per Task. Round counts do not fix training budgets: native ACE uses 96 Tasks in R1, 38 in R9, and 52 in R10. Final cost comparisons must also count analysis, planning, failed implementations, screening, serving, and evaluation, including queues and restarts. We make no end-to-end efficiency claim from task counts alone.

6 CONCLUSION

We study agents as data researchers that iteratively construct executable environments for model training. EnvOpt organizes this process around a traceable behavioral map and target-conditioned context refinement, supported by iterative environment construction. Our analyses identify gaps in behavioral understanding, feedback linkage, and environment implementation. Preliminary training results show useful checkpoints but also regressions, motivating controlled studies of sustained improvement. Further discussion and limitations appear in Appendix F.

AI USE STATEMENT

Generative AI assisted source-code and literature analysis, methodological formulation, drafting, and figure preparation. The research system uses language models for trajectory analysis, planning, program synthesis, solvability review and user simulation; generated-task rewards in the studied cohort are programmatic. Training results use existing experiment records. The offline diagnostic additionally generated and graded analysis reports from frozen public-evaluation trajectories; it did not rerun benchmark tasks or training. [TODO: The authors must verify this statement against the complete research history and review all AI-assisted claims and artifacts before submission.]

REPRODUCIBILITY STATEMENT

The research loop is specified in Section 3, construction interfaces and admission rules in Section 4, and evaluation access in Section 5. Appendix A records the training-study information needed for reproducibility; Appendix E specifies the separate report-coverage diagnostic, whose frozen aggregate data and figure/table generator are included with the manuscript. [TODO: Add the frozen implementation revision, resolved run configurations, complete artifacts, and an anonymous code supplement after the experiments are finalized.]

REFERENCES

- Victor Barres, Honghua Dong, Soham Ray, Xujie Si, and Karthik R Narasimhan. τ^2 -bench: Evaluating conversational agents in a dual-control environment. In *Forty-third International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=OC2z7iSQKa>.
- Chen Chen, Xinlong Hao, Weiwen Liu, Xu Huang, Xingshan Zeng, Shuai Yu, Dexun Li, Yuefeng Huang, Xiangcheng Liu, Wang Xinzhi, and Wu Liu. ACEBench: A comprehensive evaluation of LLM tool usage. In Christos Christodoulopoulos, Tanmoy Chakraborty, Carolyn Rose, and Violet Peng (eds.), *Findings of the Association for Computational Linguistics: EMNLP 2025*, pp. 12970–12998, Suzhou, China, November 2025. Association for Computational Linguistics. ISBN 979-8-89176-335-7. doi: 10.18653/v1/2025.findings-emnlp.697. URL <https://aclanthology.org/2025.findings-emnlp.697/>.

- Michael Dennis, Natasha Jaques, Eugene Vinitzky, Alexandre Bayen, Stuart Russell, Andrew Critch, and Sergey Levine. Emergent complexity and zero-shot transfer via unsupervised environment design. In H. Larochelle, M. Ranzato, R. Hadsell, M.F. Balcan, and H. Lin (eds.), *Advances in Neural Information Processing Systems*, volume 33, pp. 13049–13061. Curran Associates, Inc., 2020. URL https://proceedings.neurips.cc/paper_files/paper/2020/file/985e9a46e10005356bbaf194249f6856-Paper.pdf.
- Guanting Dong, Junting Lu, Junjie Huang, Wanjun Zhong, Longxiang Liu, Shijue Huang, Zhenyu Li, Yang Zhao, Xiaoshuai Song, Xiaoxi Li, Jiajie Jin, Yutao Zhu, Hanbin Wang, Fangyu Lei, Qinyu Luo, Mingyang Chen, Zehui Chen, Jiazhan Feng, Ji-Rong Wen, and Zhicheng Dou. Agent-world: Scaling real-world environment synthesis for evolving general agent intelligence, 2026. URL <https://arxiv.org/abs/2604.18292>.
- Yaxin Du, Xiyuan Yang, Zhifan Zhou, Wanxu Liu, Zixing Lei, Zimeng Chen, Fenyi Liu, Haotian Wu, Yuzhu Cai, Zexi Liu, Xinyu Zhu, WenHao Wang, Linfeng Zhang, Chen Qian, and Siheng Chen. Datamaster: Data-centric autonomous ai research, 2026. URL <https://arxiv.org/abs/2605.10906>.
- Ruishan Fang, Siyuan Lu, Chenyi Zhuang, and Tao Lin. Rods: Reward-driven online data synthesis for multi-turn tool-use agents, 2026. URL <https://arxiv.org/abs/2606.19047>.
- Runnan Fang, Shihao Cai, Baixuan Li, Jialong Wu, Guangyu Li, Wenbiao Yin, Xinyu Wang, Xiaobin Wang, Liangcai Su, Zhen Zhang, Shibin Wu, Zhengwei Tao, Yong Jiang, Pengjun Xie, Fei Huang, and Jingren Zhou. Towards general agentic intelligence via environment scaling, 2025. URL <https://arxiv.org/abs/2509.13311>.
- Hengrui Gu, Xiaotian Han, and Kaixiong Zhou. Writ: Write-read intensive trajectory synthesis for multi-turn user-facing agents, 2026. URL <https://arxiv.org/abs/2606.02908>.
- Chengsong Huang, Zifeng Wang, Rujun Han, Jun Yan, Yanfei Chen, Zoey CuiZhu, Ke Jiang, Peng Xia, Han Yu, Yufan Zhuang, Yifei Ming, Jiaqi Pan, Bhavana Dalvi Mishra, Jiabin Huang, Burak Gokturk, Tomas Pfister, and Chen-Yu Lee. Envharness: Awakening static worlds for agent learning, 2026. URL <https://arxiv.org/abs/2608.19880>.
- Hao Jiang, Gangtao Xin, Yingdi Huang, Guojie Zhu, Jiangshan Zhang, Xinyuan Lin, Yunkun Xu, Chengyu Shen, Wenlong Fei, Jiawei Li, Yujie Fu, Sichen Kang, Tingyu Xie, Yedi Hu, Jingren Zhang, Hongcheng Gao, Jianshu Zeng, Chong Chen, Chang Guo, Chao Feng, Feng Wang, Fulin Lin, Jinchao Ma, Lang Mei, Li Huang, Liyan Liu, Qing He, Shuting Tao, Siyu Mo, Xiangnan Chen, Xiaohan Yu, Xiaoyang Li, Yanheng Hou, Yanyu Wu, Zhihan Yang, Wentao Zhang, Yang Gao, and Zhao Cao. Agentomnia: Scaling agentic models for full-scenario applications, 2026. URL <https://arxiv.org/abs/2607.23124>.
- Minqi Jiang, Edward Grefenstette, and Tim Rocktäschel. Prioritized level replay. In Marina Meila and Tong Zhang (eds.), *Proceedings of the 38th International Conference on Machine Learning*, volume 139 of *Proceedings of Machine Learning Research*, pp. 4940–4950. PMLR, 18–24 Jul 2021. URL <https://proceedings.mlr.press/v139/jiang21b.html>.
- Ilia Kulikov, Chenxi Whitehouse, Tianhao Wu, Yixin Nie, Swarnadeep Saha, Eryk Helenowski, Weizhe Yuan, Olga Golovneva, Jack Lanchantin, Yoram Bachrach, Jakob Foerster, Xian Li, Han Fang, Sainbayar Sukhbaatar, and Jason Weston. Autodata: An agentic data scientist to create high quality synthetic data, 2026. URL <https://arxiv.org/abs/2606.25996>.
- Bo Liu, Simon Yu, Yiding Jiang, Ao Qu, Andrew Zhao, Zichen Liu, Junsu Kim, Zijian Zhou, Seung-gone Kim, Tongzheng Ren, Mickel Liu, Hanfei Yu, Zhaorun Chen, Weiyan Shi, Paul Pu Liang, Luke Zettlemoyer, Yejin Choi, and Natasha Jaques. Spade: Self-play in adaptive synthetic executable environments, 2026. URL <https://arxiv.org/abs/2608.19197>.
- Fanqing Meng, Lingxiao Du, Qiguang Chen, Ziqi Zhao, Haocheng Lu, Mengkang Hu, and Michael Qizhe Shieh. Rsibench-data: Benchmarking data-centric research for recursive self-improvement, 2026. URL <https://arxiv.org/abs/2607.25886>.

- Shishir G Patil, Huanzhi Mao, Fanjia Yan, Charlie Cheng-Jie Ji, Vishnu Suresh, Ion Stoica, and Joseph E. Gonzalez. The berkeley function calling leaderboard (BFCL): From tool use to agentic evaluation of large language models. In Aarti Singh, Maryam Fazel, Daniel Hsu, Simon Lacoste-Julien, Felix Berkenkamp, Tegan Maharaj, Kiri Wagstaff, and Jerry Zhu (eds.), *Proceedings of the 42nd International Conference on Machine Learning*, volume 267 of *Proceedings of Machine Learning Research*, pp. 48371–48392. PMLR, 13–19 Jul 2025. URL <https://proceedings.mlr.press/v267/patil25a.html>.
- Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. Deepseekmath: Pushing the limits of mathematical reasoning in open language models, 2024. URL <https://arxiv.org/abs/2402.03300>.
- Kou Shi, Zun Wang, Qisheng Su, Shiting Huang, Ziao Zhang, Zhen Fang, Qingnan Ren, Jin Liu, Yu Zeng, Yiming Zhao, Lin Chen, Zehui Chen, and Feng Zhao. Facet: Preserving source intent and executable state in terminal task synthesis, 2026. URL <https://arxiv.org/abs/2608.18580>.
- Xiaoshuai Song, Haofei Chang, Guanting Dong, Yutao Zhu, Ji-Rong Wen, and Zhicheng Dou. EnvScaler: Scaling tool-interactive environments for LLM agent via programmatic synthesis. In Maria Liakata, Viviane P. Moreira, Jiajun Zhang, and David Jurgens (eds.), *Findings of the Association for Computational Linguistics: ACL 2026*, pp. 8326–8357, San Diego, California, United States, July 2026. Association for Computational Linguistics. ISBN 979-8-89176-395-1. doi: 10.18653/v1/2026.findings-acl.407. URL <https://aclanthology.org/2026.findings-acl.407/>.
- Harsh Trivedi, Tushar Khot, Mareike Hartmann, Ruskin Manku, Vinty Dong, Edward Li, Shashank Gupta, Ashish Sabharwal, and Niranjan Balasubramanian. AppWorld: A controllable world of apps and people for benchmarking interactive coding agents. In Lun-Wei Ku, Andre Martins, and Vivek Srikumar (eds.), *Proceedings of the 62nd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)*, pp. 16022–16076, Bangkok, Thailand, August 2024. Association for Computational Linguistics. doi: 10.18653/v1/2024.acl-long.850. URL <https://aclanthology.org/2024.acl-long.850/>.
- Dunwei Tu, Hongyan Hao, Hansi Yang, Yihao Chen, Yu Yang, Yueqing Sun, Xingchen Liu, Furao Shen, Qi GU, Hui Su, and Xunliang Cai. Scaleenv: Scaling environment synthesis from scratch for generalist interactive tool-use agent training. In *Forty-third International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=4GX4r1CYtD>.
- Zhaoyang Wang, Canwen Xu, Boyi Liu, Yite Wang, Siwei Han, Zhewei Yao, Huaxiu Yao, and Yuxiong He. Agent world model: Infinity synthetic environments for agentic reinforcement learning. In *Forty-third International Conference on Machine Learning*, 2026. URL <https://openreview.net/forum?id=OvLnVkV14P>.
- Minrui Xu, Zilin Wang, Mengyi DENG, Zhiwei Li, Zhicheng Yang, Xiao Zhu, Yinhong Liu, Boyu Zhu, Baiyu Huang, Chao Chen, Heyuan Deng, Fei Mi, Lifeng Shang, Kingshan Zeng, and Zhi-jiang Guo. Envfactory: Scaling tool-use agents via executable environments synthesis and robust rl, 2026. URL <https://arxiv.org/abs/2605.18703>.
- An Yang, Anfeng Li, Baosong Yang, Beichen Zhang, Binyuan Hui, Bo Zheng, Bowen Yu, Chang Gao, Chengen Huang, Chenxu Lv, Chujie Zheng, Dayiheng Liu, Fan Zhou, Fei Huang, Feng Hu, Hao Ge, Haoran Wei, Huan Lin, Jialong Tang, Jian Yang, Jianhong Tu, Jianwei Zhang, Jianxin Yang, Jiayi Yang, Jing Zhou, Jingren Zhou, Junyang Lin, Kai Dang, Keqin Bao, Kexin Yang, Le Yu, Lianghao Deng, Mei Li, Mingfeng Xue, Mingze Li, Pei Zhang, Peng Wang, Qin Zhu, Rui Men, Ruize Gao, Shixuan Liu, Shuang Luo, Tianhao Li, Tianyi Tang, Wenbiao Yin, Xingzhang Ren, Xinyu Wang, Xinyu Zhang, Xuancheng Ren, Yang Fan, Yang Su, Yichang Zhang, Yinger Zhang, Yu Wan, Yuqiong Liu, Zekun Wang, Zeyu Cui, Zhenru Zhang, Zhipeng Zhou, and Zihan Qiu. Qwen3 technical report, 2025. URL <https://arxiv.org/abs/2505.09388>.
- Kingshan Zeng, Zishan Xu, Boju Zhang, Yuzhou Wu, Lingzhi Wang, Jianghao Lin, Liangyou Li, Yasheng Wang, Lifeng Shang, Xin Jiang, Weinan Zhang, Yong Yu, Qun Liu, and Weiwen Liu. What makes good agentic data? an ace lens on data generation for llm agents, 2026. URL <https://arxiv.org/abs/2608.27260>.

A INTERFACES AND PROGRAM SEMANTICS

The current interface is a complete environment program, not a mandatory World–Scene stack. A module-level `build()` returns an `Environment`. Its sampler generates all initial data and goals before rollout; an `Episode` owns each rollout’s independent mutable state. Generated modules are not imported on the host as part of paper preparation.

Table 1: Responsibilities of a reusable environment program.

Interface	Responsibility
<code>make_task(seed, parameters)</code>	Deterministically return a request and JSON payload.
<code>start(task)</code>	Create fresh episode state and actor-visible instructions.
<code>@tool(..., actor=...)</code>	Declare matching schemas and executable methods.
<code>available(name, actor)</code>	Define dynamic actor-specific visibility.
<code>next_user / user_context</code>	Supply scripted follow-ups or legitimate model-backed user context.
<code>check(task, state, trace)</code>	Evaluate requested outcomes from actual state and execution.
<code>validate(reference)</code>	Assert selected tool/reward behavior, including meaningful rejection cases.

Task serialization binds the program-bank digest, request, payload, sampling parameters, and an initial signature covering state, public instructions, visible tools and user context. Repeated sampling with identical inputs must agree. Episode state is copied; expected tool rejection restores the previous state and records the result. These checks are consistency safeguards, not a proof that arbitrary Python or all unseen task instances are correct.

Reward. A generated task returns a Boolean or programmatic `Score`. With outcome components $c_k \in [0, 1]$, mandatory guards $g_\ell \in \{0, 1\}$ and positive pre-fixed weights w_k ,

$$R_x(\tau) = \left(\prod_\ell g_\ell \right) \frac{\sum_k w_k c_k}{\sum_k w_k}, \quad \text{Success}_x(\tau) = \mathbf{1}[\forall k : c_k = 1 \wedge \forall \ell : g_\ell = 1]. \quad (8)$$

The broader framework contains optional semantic-rubric support, but the studied task-generation contract excludes it. Model-backed users and independent solvability reviewers remain distinct from programmatic training rewards. Official target graders retain their own scoring logic.

Construction and refinement in the maintained framework. The environment, task, and verifier are logical components that can share one program. Each environment has a persistent `Env Builder` session and workspace, receiving the researcher’s specification, sampling settings, generic examples, and prior implementation/review history. Different environments can be built concurrently; revisions to the same environment are serialized. Review covers the full candidate batch and previous findings. An unchanged resubmission reuses its verdict rather than resampling the reviewer; any edit requires renewed review.

Construction obeys task-count and time budgets. Training starts only when the accepted-set requirement is met: full-target settings require N task instances, whereas minimum-acceptance settings allow $N_{\min} \leq |D_t| \leq N$, without admitting unchecked triples. Already accepted instances retain their immutable versions during bounded replanning. There is no separate teacher-solution requirement, pre-training student probe, or reward-variation admission gate. These controls support implementation, not a separate methodology contribution; the historical protocol below allowed smaller positive batches.

Checks in the reported cohort. In the earlier protocol underlying the results, changed programs undergo CPU execution checks and a model review of up to three representative constructed instances (first, middle, last), not an exhaustive semantic audit. At most one code repair is permitted. Retention can reuse the accepted review when code and sampler parameters match; all new tasks still receive executable checks. Promotion checks that request, payload, initial signature and sampling semantics remain unchanged. Failed candidates and exact duplicates produce explicit shortfalls, with no unplanned backfill; positive smaller batches can enter training. These are historical study settings, not the current full-batch, persistent-builder review described in Section 4.

B STUDY VERSIONS AND EVIDENCE BOUNDARIES

The earlier manuscript described World–Scene interfaces, curated World initialization and a separate student probe. Those describe a superseded protocol, not the results reported here. The three selected runs use `persistent_environment_v5`; their initial Git trees contain only an empty-bank seed manifest under `env/`. Generic interface examples remain provided.

Maintained framework versus measured studies. The 16 September revision separates Researcher-owned interpretation from persistent Env Builders, one per environment, which receive specifications and examples rather than raw public-eval or RL evidence. It adds full-batch independent review for target alignment and behavior correctness, same-builder iterative repair, and immutable accepted-task retention across bounded replanning to meet a 96-task target before RL. Actual-exposure records distinguish intended, accepted, and completed training. These changes must not be attributed retrospectively to the earlier 4B curves. The BFCL, τ^2 , and ACE author cohorts launched on 16 September were stopped during their first construction/planning round with zero completed iterations and no formal RL; local construction and review checks do not establish benchmark gains. The earlier ACE decision-brief contract already records expected effects and previous-intervention review, whereas the earlier τ^2 /BFCL runs use their prior v5 contract.

Table 2: Frozen study identities and important differences. These are separate runs, not matched ablation arms.

	τ^2	BFCL	ACEBench-Agent
Run suffix	83b860	defa84	22571a
Launch (Beijing time)	Sep. 13, 02:25	Sep. 13, 02:25	Sep. 14, 09:46
Snapshot rounds	9, stopped	9, stopped	10, stopped
Private-eval checkpoints	R1–R9	R1–R9	R1–R10
Task bank at start	Empty	Empty	Empty
Expectation/review contract	Earlier v5	Earlier v5	Decision brief v1
RL feedback access	Earlier diagnostic views	Earlier diagnostic views	Full group queries
Pods per training job	2	2	4
Recorded episode aborts	2/6,768	33/6,472	0/5,008

Training and evaluation configuration. The shared base model is Qwen3-4B-Instruct-2507, trained with full-parameter online GRPO, RL temperature 0.7, 65,536-token context, 16,384-token response cap, task-group size eight, learning rate 3×10^{-6} , and fixed-reference KL coefficient 0.005. The recorded optimizer seed is 1234. Up to eight Tasks form one update; smaller tails are kept. Each round targets 96 Tasks, although construction failures and allocations can reduce the realized batch. The controller task configuration limits interactions separately from benchmark evaluation. Program construction is CPU work; pod counts refer to leased GPU jobs, with four GPUs per pod.

Astra serves as the data researcher through Codex. Analysis, implementation, and screening use GPT-5.5, while model-backed users use GPT-5.4. Each study has one evolution/training seed and three evaluation repeats per checkpoint on each split; these repeats do not establish cross-seed reliability. We report task-weighted success for τ^2 and BFCL and the two-category macro mean for ACE, not pass@3 or full-leaderboard scores. The three local studies begin with an empty environment bank, generic code examples, and no external grounding corpus. The retained records contain nine completed rounds each for τ^2 and BFCL and ten for ACE; incomplete or cancelled attempts add no curve points. The behavioral-coverage diagnostic is a separate offline study on frozen Qwen3-8B trajectories (Appendix E).

External baseline identity. The exported Codex–Vanilla τ^2 and BFCL runs identify their researcher model as `gpt-6-astra` in both first- and last-round Codex event logs. Their student profile is `qwen3_4b_instruct`, a distinct model role. The corresponding ACE raw artifacts were not included in this export. Repository-example and fixed-environment controls are defined in the setup; their comparison results are not yet included in the current manuscript.

Recorded complete rounds report weight changes, finite training metrics, complete policy-token capture and task-group reward normalization. This does not make their training data defect-free: aborted episodes remain part of the audit, and maintenance/restart history is retained. No cancelled

attempt or archived alternative checkpoint is substituted into a completed-round curve. The final confirmatory study should rerun a frozen, matched protocol with independent seeds.

Direct inspection of the retained rollout archives attributes 32 BFCL R2 aborts to an invalid model-backed user-context contract and one BFCL R1 abort to an empty user turn. The two τ^2 aborts are a user-tool interaction-budget exhaustion in R3 and an empty user turn in R8. These are not ordinary failed task outcomes or evidence that a task was simply too difficult. They qualify the historical training studies even though their subsequent fixed-target evaluations completed.

B.1 ENVIRONMENT IMPLEMENTATION CASE

Section 3.3 uses the retained `retail_exchange_batch` candidate from the 16 September τ^2 construction cohort. The original specification’s success example requested different replacements for two identical mugs. The candidate’s sampler selected a replacement solely from the product group, assigning the same preferred or fallback option to every occurrence in that group. All six inspected tasks used identical replacement targets for their repeated originals. The initial review had accepted the candidate; a subsequent contract audit confirmed the missing generator branch and a related verifier mismatch for distinct-target duplicates.

The audit distinguishes generator support from batch coverage: a small batch need not exhibit every supported optional variation. Here, source inspection established that the required variation could not be sampled. A later repaired copy supported distinct targets, but does not change the original candidate or provide a downstream RL measurement. The cohort stopped before formal training. A local case record retains the specification example, source-code excerpt, historical review findings, observed batch counts, and source hashes; paper preparation only read those records and performed no new environment execution or model review.

B.2 BENCHMARK PROTOCOLS

τ^2 -Bench. Each half has 25 airline, 57 retail and 57 telecom tasks. The revised fixed-seed partition stratifies airline/retail business-action labels and telecom issue/persona labels. Its baseline was repartitioned from existing real evaluations without resampling responses; some earlier private-eval-side records required reconstruction of static tool/schema wrappers, which is disclosed in their provenance. Overall is task-weighted, not an unweighted mean of domains. The evaluation policy is greedy and the user is GPT-5.4 at temperature zero.

Evaluation of each checkpoint on the private eval set uses a replay repair that preserves the live behavior of unknown/rejected tools instead of allowing an exception to abort final-state reconstruction. Accepted private-eval records use the corrected judge configuration and replay-v2 lineage; failed and superseded v1 evaluations are excluded. The historical private-eval baseline predates the replay repair and is explicitly marked non-comparable in the dashboard registry. Figure 4 connects it for visual continuity and shows the stored-score difference; this is not a matched-protocol gain. We therefore do not report a matched-protocol gain estimate or paired interval for this benchmark.

BFCL. The fixed public/private eval halves contain 100 tasks each from Base, Long Context, Missing Function and Missing Parameter. Legacy framework identifiers such as `bfcl_v3` are adapter names, not evidence that a full leaderboard suite was evaluated. The matched private-eval base comes from the recorded September 6 collection with the same task membership, model, repeats and evaluation/serving configuration. The paper reports only these four multi-turn subsets.

ACEBench-Agent. The English Agent release is pinned to official source revision `56dd66c`. Its 20 multi-step and 30 multi-turn tasks are split by category and tool-class strata into equal public/private eval halves (10+15 each), with seed 1234. We report the unweighted mean of the two category scores, matching the metric definition used in SPADE, not its complete evaluation protocol.

The studied run uses `acebench_native_fc_v1`: native structured tool calls and full append-only assistant/tool history, with the native business tools, task initialization, GPT-5.4 user, 40-event budget and official end-state grading. The audited schema projection corrects the documented required-field typo for `reserve_flight`; malformed arguments are not repaired to make episodes succeed. The 4B policy uses temperature 0.001 and a 1,000-token output budget. Multi-

step outer prompts retain the wrapper’s Chinese compatibility setting while tasks/tools remain English; multi-turn prompts are English. These transport, history, user-model and serving details preclude presenting the scores as an exact SPADE reproduction.

The older September 13 ACE textual-interface study is not spliced into the native-FC curve. Its archived alternate training attempt is also excluded. Native private-eval measurements cover all ten completed checkpoints. The later sweep reuses R1–R4 results and adds R5–R10 on the same 25 private-eval tasks with three repeats and the same native-FC protocol. R6/R8 initially failed during remote toolchain detection before inference; the completed recovery results are included and the failed-start records are retained in provenance. Training was stopped during R11, which has no registered complete checkpoint and is excluded.

B.3 SNAPSHOT AND STATISTICAL CONSTRUCTION

Private results at public-selected checkpoints. BFCL selects R9, scoring 42.08% versus 25.42% for its matched base model (+16.67 percentage points). ACEBench-Agent selects R3, scoring 52.22% versus 41.11% (+11.11 points); its paired-task interval includes zero. ACE’s higher private score of 56.11% at R7 does not change selection, and its R10 score is 51.11%. τ^2 selects R5 with 39.33% private success, falling to 34.05% at R9. Its historical 31.89% private baseline predates the replay repair and does not support a matched-protocol gain. AppWorld selects R9 from public TGC, scoring 22.22% on private Normal versus 8.93% at Base. The supplied AppWorld series is displayed through R10; evaluation repeats are unspecified. Figure 4’s vertical guides mark public-selected checkpoints.

The original snapshot at 14 September 2026, 16:04:42 UTC remains unchanged. A supplementary snapshot at 17:43:25 UTC (15 September 01:43:25 Beijing time) adds ACE R5–R10 private-eval results and R10 public-eval/training records. It retains the other two studies exactly, yielding nine completed rounds each for τ^2 /BFCL and ten for ACE. All included checkpoints have private-eval results. Rebuilding the paper does not query or update source experiments.

Each plotted evaluation was independently recounted from stored per-task three-repeat outcome groups and checked against saved summary metrics. The updated snapshot records 218 source-file hashes, the prior snapshot’s hash, run/job/revision identities, actual task allocations, training audit fields, and links to baseline and operator-only private-eval collections. The update checks that earlier ACE measurements and training records are unchanged; new private-eval records match registered checkpoints and retain the fixed task membership. It contains outcome summaries, not a new evaluation. The paper scripts neither import generated environment modules nor invoke training, serving, grading or controller actions.

Selection uses the earliest maximum public-eval mean over R0 and the included completed checkpoints, with numerical ties rounded to twelve decimal places. It is an explicitly applied analysis rule, not a claim of preregistration. For paired private-eval differences, each bootstrap draw samples tasks with replacement within their fixed subset, retains all three repeats for each sampled task, and applies the original aggregation: task-weighted for BFCL and a two-category macro mean for ACE. We use 10,000 draws with bootstrap seed 20260914. The τ^2 baseline mismatch excludes its paired baseline comparison.

C WITHIN-BENCHMARK EVOLUTION

D CONFIRMATORY COMPARISONS STILL NEEDED

Controls should share model initialization, target-information access and evaluation protocols. Compare both matched training episodes and matched total research budgets; count failed implementations, reviewer requests and all serving/evaluation overhead. Additional evidence should include independently audited accepted and rejected tasks, one traced intervention from source case to code change and subsequent outcomes, and repeated evolution seeds. Independent private-eval tasks reserved after method freeze are needed for a final confirmatory result.

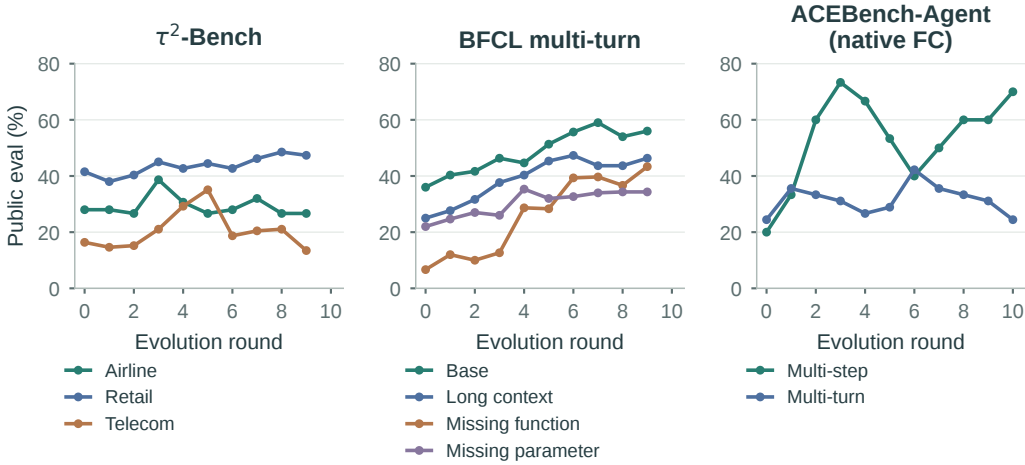


Figure 6: Fixed public-eval-subset trajectories. These use the benchmark’s fixed labels, not the data researcher’s changing analysis-domain names. BFCL’s Missing Function subset improves substantially; the later τ^2 decline is particularly visible in Telecom. Native ACE’s early gain is concentrated in Multi-Step. The figure describes associations within the recorded runs, not attribution to a particular environment revision.

Table 3: Unmeasured controls; no synthetic scores are supplied.

Control	Question isolated
Fixed library, fresh Tasks	Does later research improve over resampling and training alone?
Sampler/allocation only	Does executable program revision add value?
No maintained experience	Does accumulated, corrected research experience help?
Stale target evidence	Does fresh behavioral feedback change useful decisions?
No stable requirement map	Does explicit task-linked coverage improve retention or targeting?
Whole-program generation without persistence	Does retaining and revising assets improve cost or transfer?

E BEHAVIORAL-COVERAGE DIAGNOSTIC

Source-grounded construction and refinement. Equations 4–6 describe the coverage-checked variant evaluated here, not a retroactive change to the training-study implementation. Same-task repeats are analyzed together; oversized packs are split into complete episodes. The ledger retains source-backed statements and their task contexts independently of successive drafts. Required behavior remains distinct from observed mastery, and recovered errors from unresolved failures. A source ID verifies ancestry, not semantic retention; unavailable analyses remain missing evidence.

Hierarchical merging and source-scoped review form a report whose shared premises are then factored under a word target. Checks compare its claims with their cited mechanisms and public contexts. Local edits repair explicit conflicts; residual completion consolidates only unexpressed relations, retaining relevant actor, authorization, scope, ordering, and recovery conditions. Equivalent content is matched to a literal existing passage rather than appended again. Incidental names and repeated narratives may be removed, but operative distinctions are retained as separate branches. The final variant reuses its own compact draft, local repairs, and internal missing-source findings; it does not receive evaluation references, judge scores, or the native report’s prose. Internal checks remain fallible.

Task-linked measurement in the maintained framework. Separately from report construction, the data researcher maintains requirement definitions and sets J_c of real public-eval-task IDs. The

controller computes their performance from recorded outcomes:

$$s_t(c) = \frac{1}{|J_c|} \sum_{i \in J_c} \frac{1}{K} \sum_{k=1}^K y_{t,i,k}, \quad (9)$$

where K is the number of evaluation repeats and $y_{t,i,k}$ the recorded outcome. Changed definitions or memberships require recomputing baseline and current results on the same membership. Groups may overlap; composite-task success does not isolate individual skills. This measurement is not inferred from summary frequency or used as the diagnostic’s semantic-coverage score. Source-to-case queries expose complete permitted interactions and cross-checkpoint histories; internal merge ancestry is retained without promising arbitrary intermediate-node browsing.

Inputs and comparison. We freeze complete initial public-evaluation corpora: BFCL has 400 indexed tasks and 1,200 trajectories, τ^2 has 139 tasks and 417 trajectories, and ACEBench-Agent has 25 tasks and 75 trajectories. Each task has three evaluation repeats; these are not independent analysis seeds. These Qwen3-8B reasoning trajectories are separate from the Qwen3-4B training curves. Public/private denotes researcher access, not the benchmark’s official release status. The native analyst uses `gpt-6-astra/high` in a fresh session with a minimal task-and-schema instruction, a 60-minute ceiling, and ordinary coding and search tools. It can choose its own strategy and has no imposed report word limit. Its three saved reports are fixed for all candidate comparisons.

The refined-map generator uses `gpt-5.5/high`, complete public task evidence and its own intermediate records. It receives neither reference labels nor the agent report’s prose; only the latter’s word count sets a soft 85% length target. The final reports are 3,637, 3,775, and 2,596 words; τ^2 is shorter than the agent but does not meet that 85% target. Same-task repeats are grouped, oversized groups are split into complete episodes, and shared definitions are deduplicated without truncating observations. The initial ACE scoped-review pilot supplies system rules and the first user request as extra context; later BFCL/ τ^2 scoped reviews supply all public requests. Leaf extraction and review receive complete episodes in all three corpora.

Fixed automatic evaluation. References are independently generated from public trajectories with GPT-5.5/high, before seeing the candidate reports, and retain source quotations. The same model grades anonymized report pairs as full, partial, missing, or contradicted. Full requires preserving the mechanism and its consequential scope, condition, actor, or order; mere category overlap is insufficient. Only full contributes to the strict score. First average reference units within each indexed task, then average tasks; the supplemental metric additionally averages fixed subset–family groups equally. Repeat-level evidence does not increase the independent task count. The reference inputs, grading prompt, batch size (24), model settings, blind seed, and aggregation are unchanged across candidates. Literal supporting quotations are checked mechanically.

Corpus	Agent original	Agent paired	Refined map	Words (agent / map)
BFCL	71.29 / 47.65	61.49 / 39.40	88.08 / 71.31	5,134 / 3,637
τ^2	86.24 / 69.92	78.66 / 61.88	97.50 / 80.48	4,283 / 3,775
ACE-Agent	88.38 / 76.57	82.46 / 67.46	98.24 / 88.36	3,337 / 2,596

Table 4: Task / failure coverage (%) and report lengths. “Original” and “paired” grade the same native report in different judging contexts, not different native generations.

Corpus	Agent original	Agent paired	Refined map
BFCL	67.68 / 49.77	57.81 / 41.23	86.00 / 69.41
τ^2	70.95 / 55.27	58.17 / 49.83	95.05 / 74.02
ACE-Agent	90.73 / 80.11	86.02 / 74.34	98.12 / 90.53

Table 5: Subset–family-balanced task / failure coverage (%). Families are a fixed diagnostic taxonomy, not an exhaustive ontology of benchmark failures.

Evaluation provenance and missingness. Sections 3.1 and 5.3 use the same final paired-evaluation results. Earlier scores are retained as historical results, not pooled with that evaluation or interpreted as a same-input repeatability test. Saved configurations retain the judge model, prompt,

reasoning level, batch size, and blind seed; the paired comparison report changes, as do some eligible scoring units. The score difference therefore does not isolate judge instability, and a downward shift in the baseline is not a method contribution. Both current and original agent scores are below the refined map’s four coverage metrics in every corpus. Historical grades have slightly different questionable or unscored units and are not paired causal effect estimates. Within each comparison the methods share the same eligible units. BFCL scores 8,327 units and ACE 538; τ^2 retains 3,191 of 3,200. Respectively, 75, 14, and 2 units are marked questionable and excluded jointly. Original reference annotations are unavailable for 7, 13, and 1 trajectories. One bounded format-retry pass and the pre-existing valid-paired-unit recovery rule are used; an interrupted τ^2 retry is concluded from saved answers without another scoring pass.

Astra/high provides a separate, disagreement-enriched spot audit. BFCL, τ^2 , and ACE have 22/24, 23/24, and 22/24 evaluable items: 21/23/22 references are supported and one BFCL reference is uncertain. Exact first/second-rater label agreement for agent/map is 15/22 versus 18/22, 20/23 versus 20/23, and 15/22 versus 20/22. These samples are not global accuracy estimates or human gold.

Illustration and counterexamples. In the ACE recovery example discussed in Section 5.3, a successful purchase is followed by a capacity-blocked notification. The agent retrieves messages but reaches its turn limit without obtaining a deletion target, deleting, or retrying. The native report mentions inbox-cleanup truncation; the refined map preserves the recovery dependency and distinguishes interactive authorization from preset deletion. Both graders support partial versus full coverage. This is not evidence of deliberate abandonment or of successful recovery.

The method still loses information: it compresses a BFCL card-number-as-card-ID error and its recovery into a generic identifier warning, and an ACE repeated date-argument error after TypeError feedback into a parameter-error category. For BFCL prefix-only deletion of a 31-file list, the second grader downgrades the map’s “Drafts partial deletion” from full to partial. A separate purposeful four-item BFCL case audit retains three evaluable results and one format failure; it does not modify primary scores or enter the main audit counts.

Compute and interpretation. Generation costs include reused source extraction and parent stages, not just the final compression call. The map uses substantially more computation; input includes cached tokens and output includes reasoning tokens. Unknown usage is not zero, and token totals across different models are not monetary costs. Reference construction, scoring and audit requests are evaluation overhead preserved separately in the study logs.

Corpus / method	Input (M)	Cached (M)	Output (M)	Unknown usage
BFCL / Agent	0.876	0.766	0.017	0
BFCL / Map	43.241	0.503	8.949	20
τ^2 / Agent	1.327	1.213	0.016	0
τ^2 / Map	16.192	0.593	4.077	0
ACE-Agent / Agent	1.462	1.363	0.012	0
ACE-Agent / Map	1.049	0.047	0.591	0

Table 6: Recorded generation tokens in millions; cached input is a subset of input. Unknown-usage requests are disclosed separately. These totals describe the studied prototype, not an efficiency-matched comparison.

All candidate versions and negative results are retained. The original simple merge did not uniformly outperform the agent; scoped expansion improved detail but grew long, while the final refinement restored only semantic residuals. The final candidate does not dominate every intermediate version on every metric. These corpora were used for method development, not an unseen confirmation study. The diagnostic supports reduced information loss in this setting, not semantic losslessness, a same-model/same-budget causal attribution, the independent value of source lookup, or downstream RL improvement. The refinement is not retroactively attributed to the historical training runs.

Provenance of the motivating cases. Figure 2 reads the final native strict-coverage grades directly from the same frozen results as Figure 5. The archived BFCL case audit uses public trajectory `efefelb89e2fb4e8382f9a5f0c230912`, preserved locally with the diagnostic sources. Six successful file-deletion calls target the first six entries of the 31-file listing; the other 25 entries are not targeted. Two subsequent directory-removal calls report a missing path, yet the final reply

claims complete cleanup. The audit identifies an omitted execution pattern, not an independently verified final filesystem state or a fully repaired summary. The archived native excerpt is a generic filesystem finding, not a diagnosis of this specific trajectory. The complete-report case audit judges the prefix-only mechanism missing under both raters; the excerpt alone does not establish absence. Its verbatim text and report fingerprint are retained in `bfcl_prefix_report_excerpt.json` beside the local case and audit. The separate ACE R9 record audit uses the frozen 4B results: 48 planned tasks, 38 trained tasks, 304 episodes, five updates, zero recorded episode aborts, and 45.56% public-eval score. The purchase-workflow revision's allocation is 10 planned, zero accepted, and unavailable, with no duplicates or batch-alignment drops. The case is a plan-exposure record audit, not an unlinked-feedback control; it neither attributes a score change to missing data nor rules out exposure to earlier versions. The aggregate scores and original cases are summarized in `data/evidence_gaps_20260918.json`; no new training or judging was performed for these illustrations.

F DISCUSSION AND LIMITATIONS

EnvOpt centers recursive environment research on two questions: what behavior needs training, and what the previous intervention actually delivered. Its behavioral map and target-conditioned context refinement make the corresponding evidence inspectable. They do not remove the modeling obligation of from-scratch generation: observed traces underdetermine hidden state and unobserved actions. A program can execute correctly while training a simplified or irrelevant decision. Empty-library initialization removes a supplied business environment, not the prior knowledge in language models, interface code, or generic examples.

Source-linked analysis is an investigation aid rather than an objective capability detector. Existing audits identify unsupported prerequisites inferred from the agent's own errors, successful traces described too favorably, and changing domain granularity across rounds. The offline coverage diagnostic supports more compact, informative reports, but uses different analyst backbones and unequal computation; its corpora were used for method development and its automatic grades are not human ground truth. It does not establish lossless analysis or downstream training benefits. Stable task membership prevents some counting ambiguities, but composite-task scores do not isolate individual skills. Likewise, aligned goal data and bounded solvability review do not guarantee a correct verifier; author and reviewer errors can be correlated.

The training results are preliminary, single-seed and protocol-qualified. They lack fixed-library and budget-matched adaptive controls; historical training anomalies and maintenance interventions are disclosed rather than erased. Public-eval-conditioned environment research can overfit its feedback. Although private-eval data are excluded from the data researcher, repeated operator inspection of checkpoint private evaluations limits claims of a pristine final holdout. A final confirmatory study should freeze the method and selection rule before using a fresh private eval set. Most importantly, later regressions show that discovering a useful checkpoint and sustaining reliable self-improvement remain different research outcomes.